Part 1: Software Architecture Selection and Justification

1. Introduction

The system described is a restaurant chain application designed to display the ten most popular food items sold daily, along with menus for breakfast, lunch, dinner, and beverages. The application will initially be developed as a web-based system and later extended to support mobile applications for both Android and iOS platforms. The system relies on distributed database servers located in each state, where transaction data is stored and processed.

2. Selected Architecture

The selected architectural pattern for this system is the **3-tier client-server architecture**. This architecture separates the system into three primary layers: the presentation layer (client), the application layer (server), and the data layer (database). This separation enables modular design, scalability, and easier maintenance.

3. Justification of Architecture

3.1 Deployment Considerations

The 3-tier client-server architecture supports a distributed deployment model, which is essential for this system. The client layer consists of web browsers and mobile devices that interact with the system via HTTP/HTTPS protocols. The application layer is hosted on cloud-based servers (such as AWS or Azure), where backend services handle business logic, data processing, and database communication. The data layer consists of distributed state database servers that store transaction data.

This architecture allows each layer to be deployed independently. For example, application servers can be scaled horizontally to handle increased user traffic, while database servers remain distributed across states. Additionally, cloud deployment ensures high availability and reliability.

3.2 Usability Considerations

From a usability perspective, the architecture provides a clear separation between the user interface and system logic. The web application and future mobile applications act as clients, offering intuitive interfaces for users to access menus and popular items. Since the application logic is centralized in the backend server, updates and improvements can be made without requiring users to reinstall applications. This ensures consistency across platforms and enhances user experience.

3.3 Quality Attributes

The selected architecture supports several key quality attributes:

- **Scalability:** The system can handle increasing numbers of users by scaling the application servers in the cloud.
- **Performance:** Caching mechanisms (e.g., Redis) improve response time by storing frequently accessed data, such as popular food items.
- **Security:** Sensitive data is managed within the backend and database layers, reducing exposure to clients.
- **Maintainability:** The separation of concerns allows developers to modify one layer without significantly impacting others.
- **Modifiability:** The system can easily be extended to support mobile applications and additional features in the future.

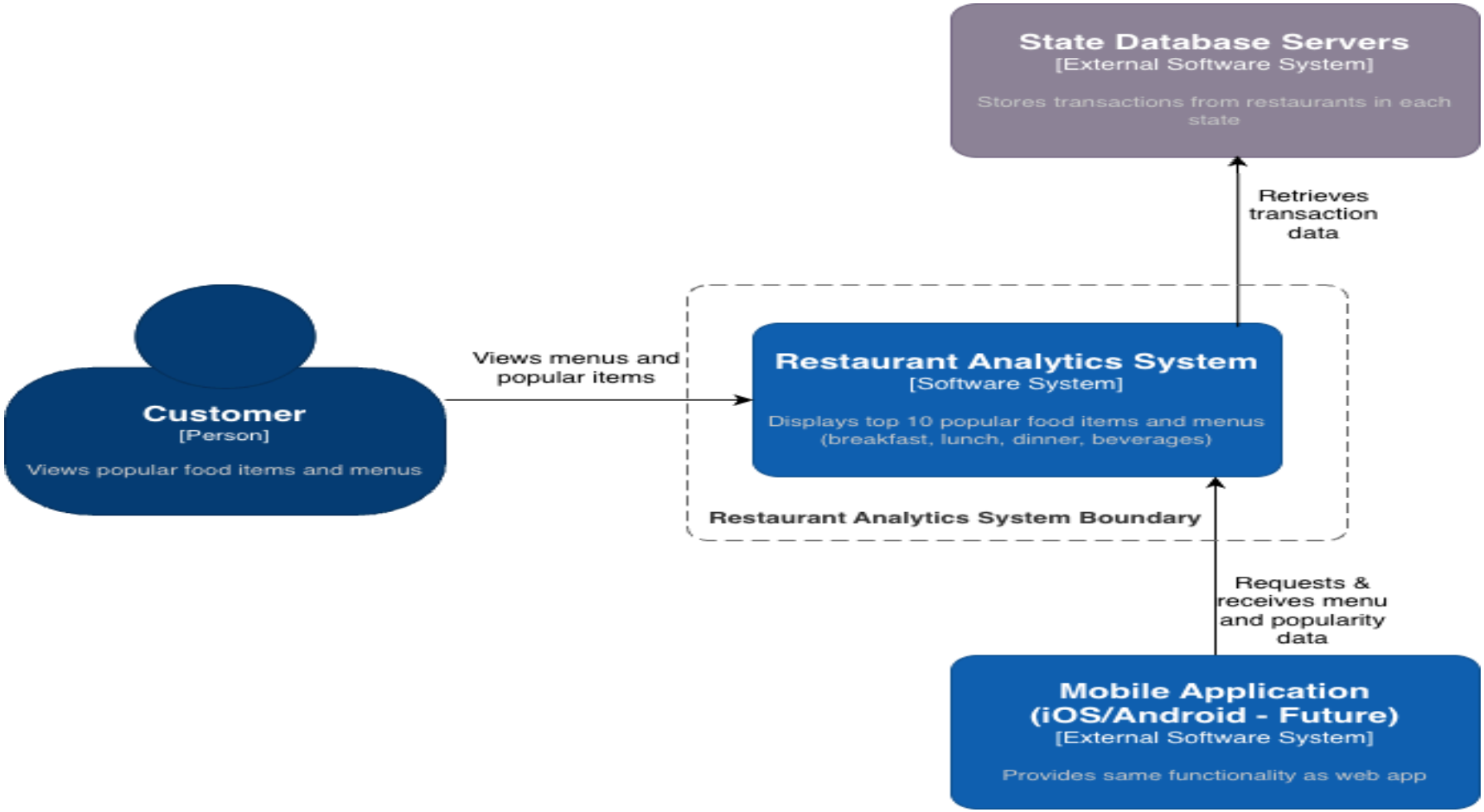
4. Conclusion

The 3-tier client-server architecture is the most suitable choice for this system due to its ability to support distributed data, multiple client types, and scalable deployment. It provides a strong foundation for both current requirements and future expansion, ensuring high performance, maintainability, and usability.

Part 2: Software Architecture Diagrams (C4 Model)

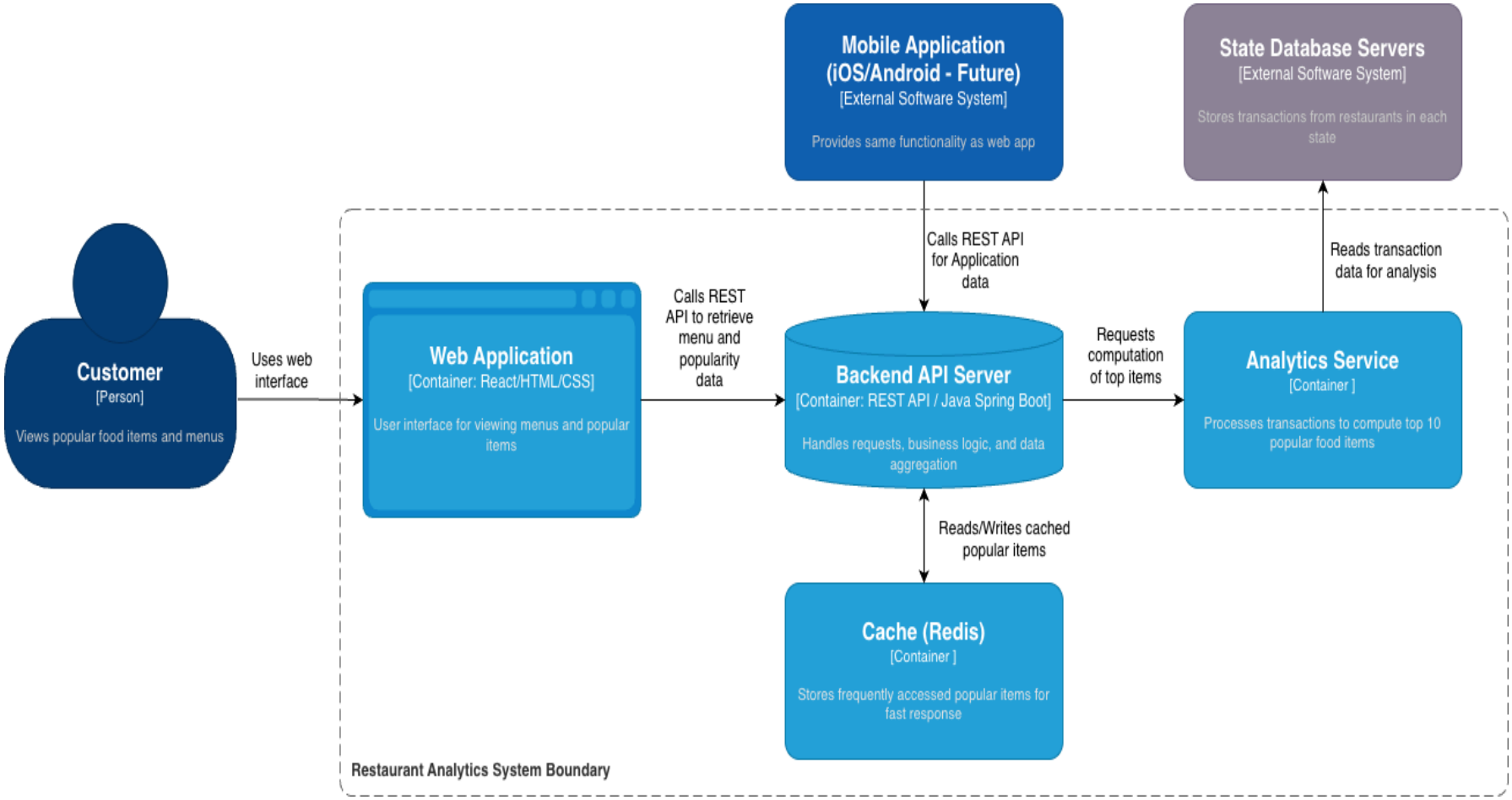
2.1 Context Diagram

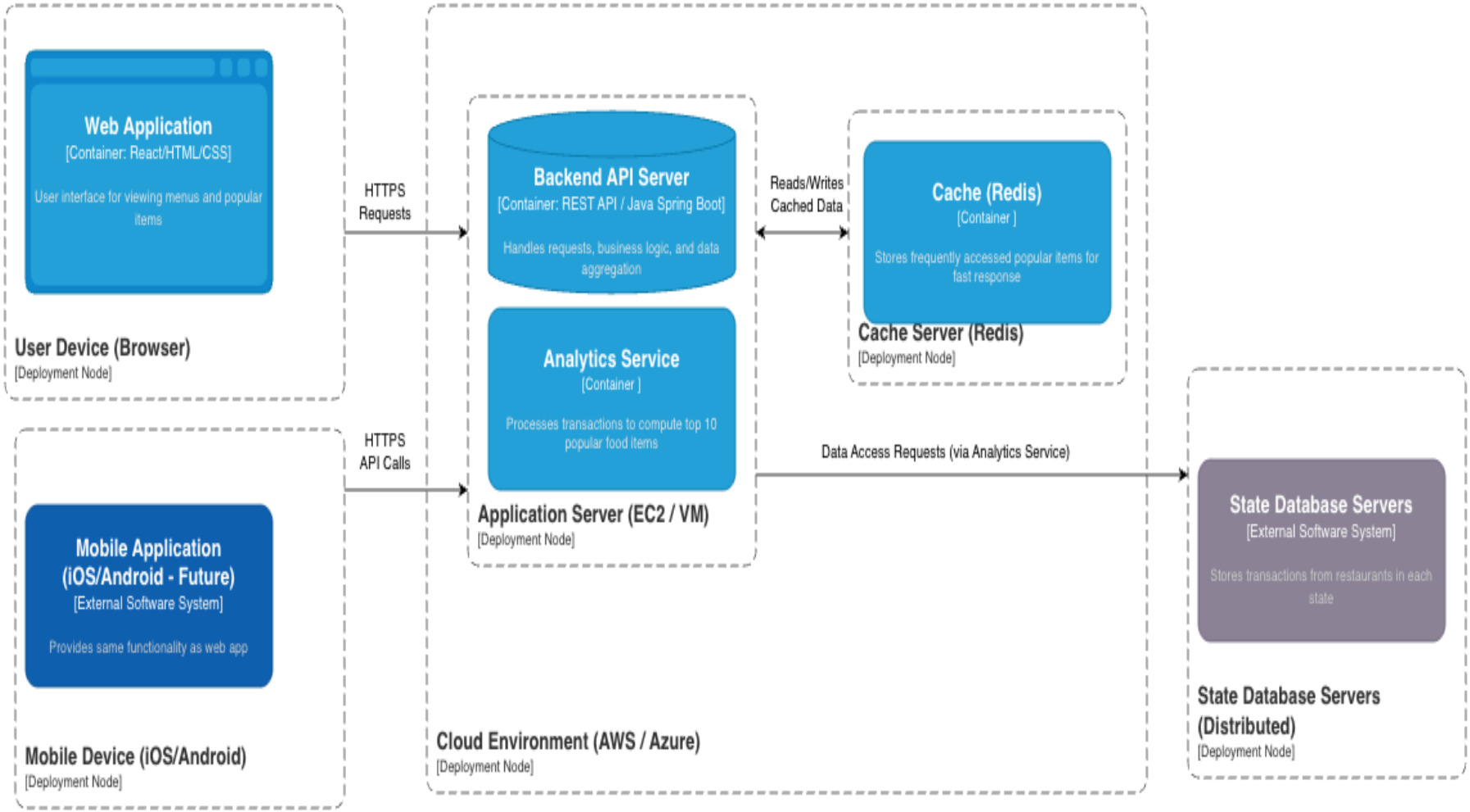
The context diagram illustrates the high-level interaction between the system, its users, and external systems. It identifies the primary user (Customer), the Restaurant Analytics System, and external dependencies such as state database servers and mobile applications. This diagram provides an overview of the system's scope and its interactions with external entities.



2.2 Container Diagram

The container diagram breaks down the system into its main functional components. The system consists of a web application for user interaction, a backend API server for handling business logic, an analytics service for processing transaction data, and a caching layer to improve performance. External systems such as mobile applications and distributed databases interact with these containers.



2.3 Deployment Diagram

The deployment diagram illustrates how the system is deployed across different infrastructure nodes. It shows user devices (browser and mobile), cloud-based application servers, cache servers, and distributed database servers. The diagram also highlights communication between nodes using protocols such as HTTPS and internal service calls, providing a clear view of runtime interactions and infrastructure layout.